Import Libraries

We only need 3 libraries:

- **pandas** => to load and work with the data
- **matplotlib** => to make charts
- **seaborn** => to make nicer charts

In [29]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
```

Load the Dataset

In [31]:
```
df = pd.read_csv("C:/Users/kunwa/Downloads/archive (5)/Loan_default.csv")
df.shape
```

Out[31]:
```
(255347, 18)
```

Data

In [26]:
```
df.head()
```

Out[26]:

	LoanID	Age	Income	LoanAmount	CreditScore	MonthsEmployed	NumCreditLines	Ir
0	I38PQUQS96	56	85994	50587	520	80	4	
1	HPSK72WA7R	69	50432	124440	458	15	1	
2	C1OZ6DPJ8Y	46	84208	129188	451	26	3	
3	V2KKSFM3UN	32	31713	44799	743	0	3	
4	EY08JDHTZP	60	20437	9139	633	8	4	

In [27]:
```
df.tail()
```

Out[27]:

	LoanID	Age	Income	LoanAmount	CreditScore	MonthsEmployed	NumCreditLir
255342	8C6S86ESGC	19	37979	210682	541	109	
255343	98R4KDHNND	32	51953	189899	511	14	
255344	XQK1UUUNGP	56	84820	208294	597	70	
255345	JAO28CPL4H	42	85109	60575	809	40	
255346	ZTH91CGL0B	62	22418	18481	636	113	

In [28]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 255347 entries, 0 to 255346
Data columns (total 18 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   LoanID          255347 non-null  object
 1   Age             255347 non-null  int64
 2   Income          255347 non-null  int64
 3   LoanAmount      255347 non-null  int64
 4   CreditScore     255347 non-null  int64
 5   MonthsEmployed  255347 non-null  int64
 6   NumCreditLines  255347 non-null  int64
 7   InterestRate    255347 non-null  float64
 8   LoanTerm        255347 non-null  int64
 9   DTIRatio        255347 non-null  float64
 10  Education       255347 non-null  object
 11  EmploymentType  255347 non-null  object
 12  MaritalStatus   255347 non-null  object
 13  HasMortgage     255347 non-null  object
 14  HasDependents   255347 non-null  object
 15  LoanPurpose     255347 non-null  object
 16  HasCoSigner     255347 non-null  object
 17  Default         255347 non-null  int64
dtypes: float64(2), int64(8), object(8)
memory usage: 35.1+ MB
```

Check for Missing Values

Missing values can cause problems in our model, so we need to know where they are.

In [32]:
```
#for missing values
df.isna().sum()
```

Out[32]:
```
LoanID            0
Age               0
Income            0
LoanAmount        0
CreditScore       0
MonthsEmployed    0
NumCreditLines    0
InterestRate      0
LoanTerm          0
DTIRatio          0
Education         0
EmploymentType    0
MaritalStatus     0
HasMortgage       0
HasDependents     0
LoanPurpose       0
HasCoSigner       0
Default           0
dtype: int64
```

Checking Duplicate Rows

In [12]:
```
duplicates = df.duplicated().sum()
print(f'Number of duplicate rows: {duplicates}')
```

```
Number of duplicate rows: 0
```

Separating Numerical and Categorical Columns

This helps us analyze each type differently.

In [13]:
```
# Numerical columns
num_cols = ['Age', 'Income', 'LoanAmount', 'CreditScore',
            'MonthsEmployed', 'NumCreditLines', 'InterestRate',
            'LoanTerm', 'DTIRatio']

# Categorical columns
cat_cols = ['Education', 'EmploymentType', 'MaritalStatus', 'LoanPurpose']

# Binary columns (Yes/No)
bin_cols = ['HasMortgage', 'HasDependents', 'HasCoSigner']

print('Numerical columns :', num_cols)
print('Categorical columns:', cat_cols)
print('Binary columns     :', bin_cols)
```

```
Numerical columns : ['Age', 'Income', 'LoanAmount', 'CreditScore', 'MonthsEm
ployed', 'NumCreditLines', 'InterestRate', 'LoanTerm', 'DTIRatio']
Categorical columns: ['Education', 'EmploymentType', 'MaritalStatus', 'LoanP
urpose']
Binary columns     : ['HasMortgage', 'HasDependents', 'HasCoSigner']
```

Descriptive Statistics for Numerical Columns

This shows count, mean, min, max, etc. for each number column.

In [15]:
```
# Basic statistics: count, mean, std, min, max, quartiles
df[num_cols].describe().round(2)
```

Out[15]:

	Age	Income	LoanAmount	CreditScore	MonthsEmployed	NumCreditLines	Inter
count	255347.00	255347.00	255347.00	255347.00	255347.00	255347.00	25
mean	43.50	82499.30	127578.87	574.26	59.54	2.50	
std	14.99	38963.01	70840.71	158.90	34.64	1.12	
min	18.00	15000.00	5000.00	300.00	0.00	1.00	
25%	31.00	48825.50	66156.00	437.00	30.00	2.00	
50%	43.00	82466.00	127556.00	574.00	60.00	2.00	
75%	56.00	116219.00	188985.00	712.00	90.00	3.00	
max	69.00	149999.00	249999.00	849.00	119.00	4.00	

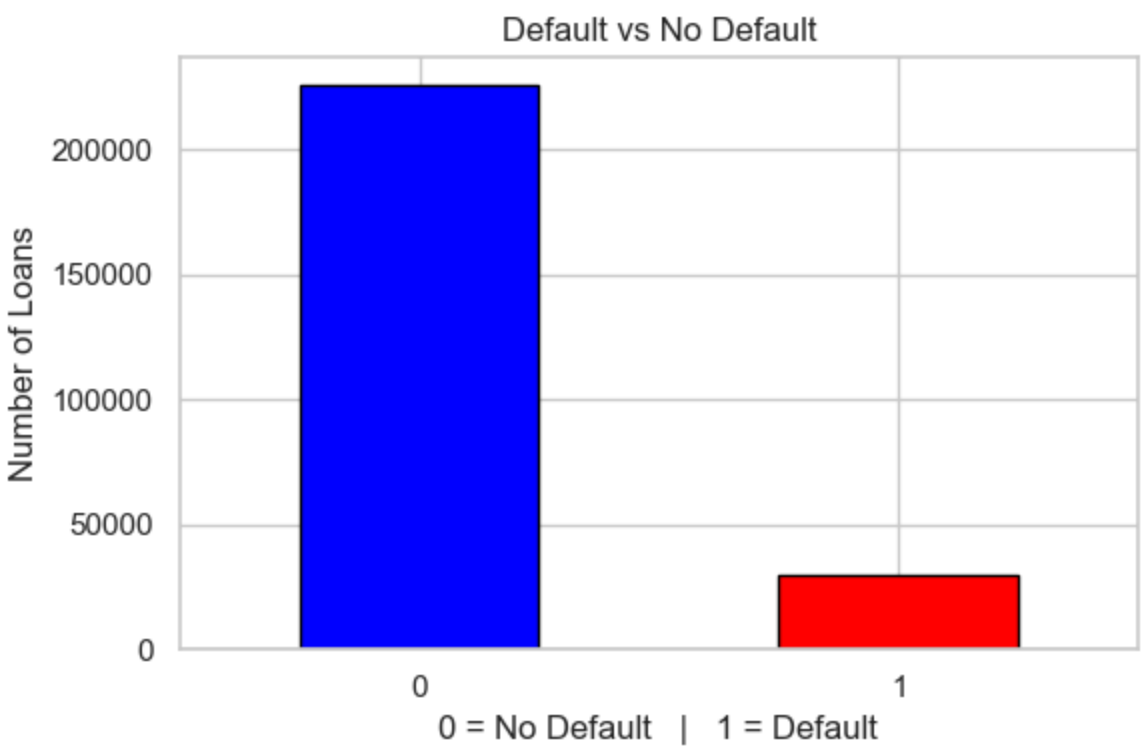
Target Variable, How Many Defaulted?

We check how many loans ended in default (1) vs. no default (0).

In [16]:
```
# Count how many defaulted vs not
print('Default Value Counts:')
print(df['Default'].value_counts())
print()
print(f"Default Rate: {df['Default'].mean() * 100:.2f}%")
```

```
Default Value Counts:
0    225694
1     29653
Name: Default, dtype: int64

Default Rate: 11.61%
```

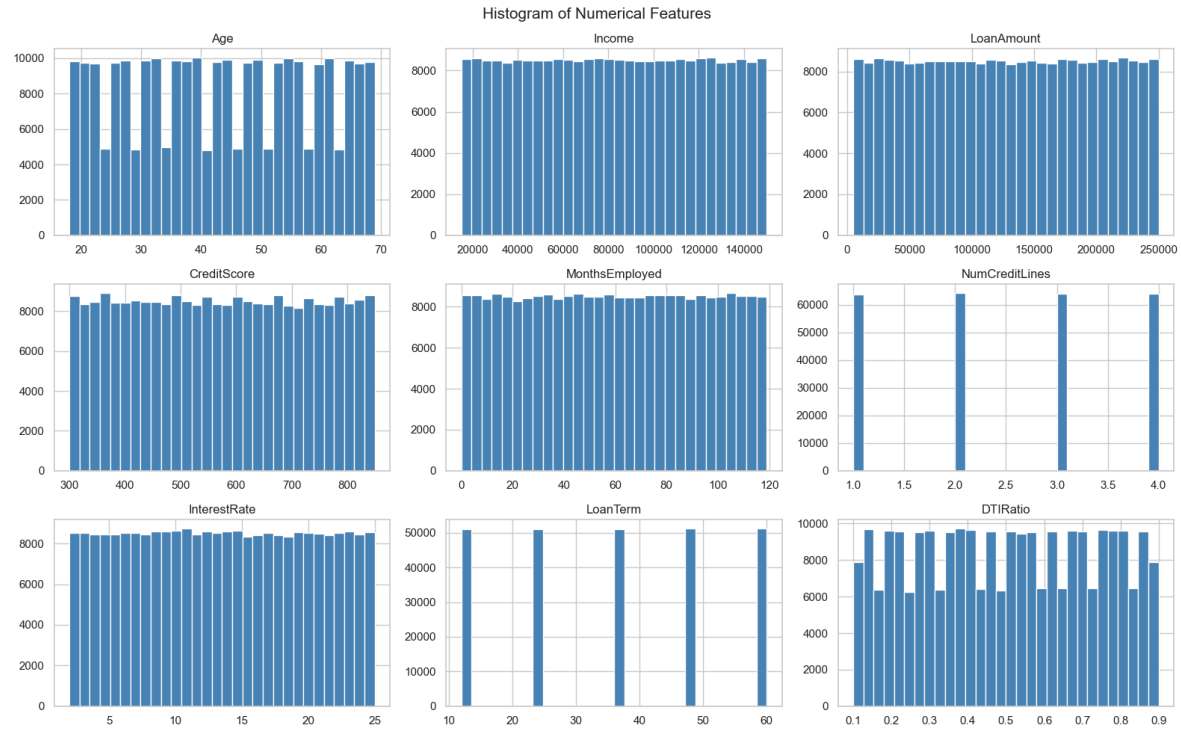
In [34]:
```
# Bar chart for Default
plt.figure(figsize=(6, 4))
df['Default'].value_counts().plot(kind='bar', color=['blue', 'red'], edgecolor
plt.title('Default vs No Default')
plt.xlabel('0 = No Default   |   1 = Default')
plt.ylabel('Number of Loans')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



Distribution of Each Numerical Column

A histogram shows how the values are spread out.

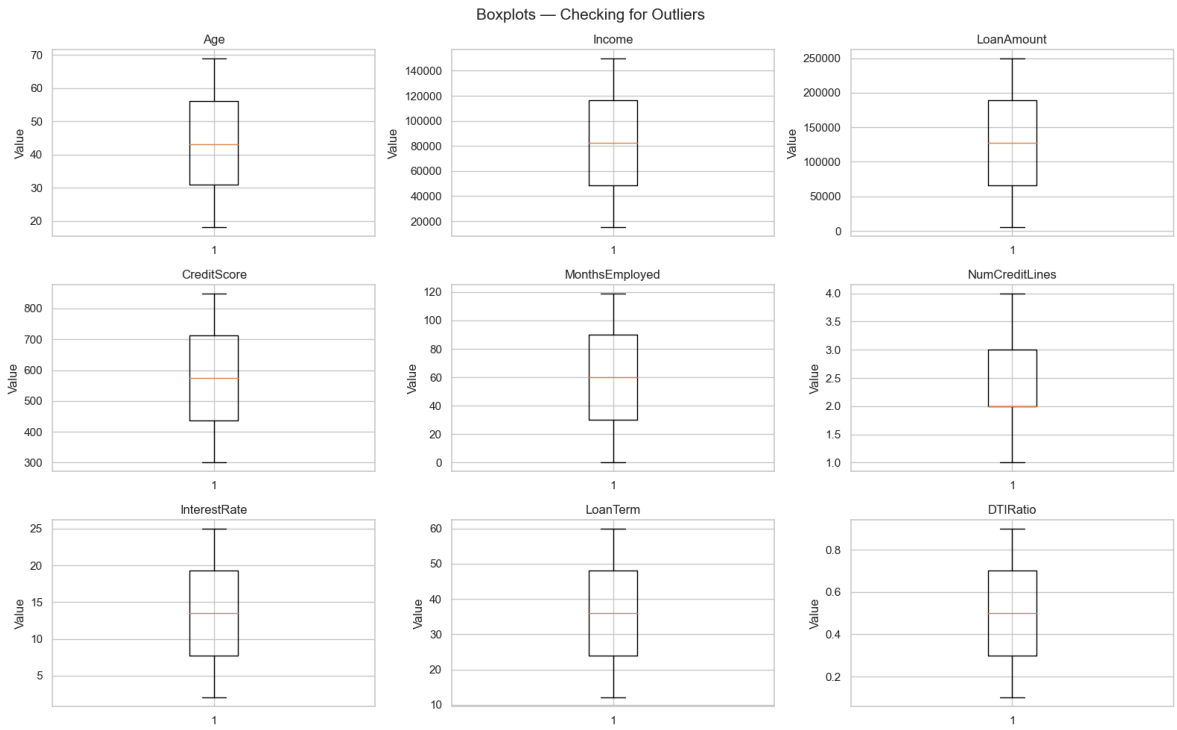
In [35]:
```
# Plot histogram for each numerical column
df[num_cols].hist(bins=30, figsize=(16, 10), color='steelblue', edgecolor='whi
plt.suptitle('Histogram of Numerical Features', fontsize=15)
plt.tight_layout()
plt.show()
```



Boxplots, Spotting Outliers

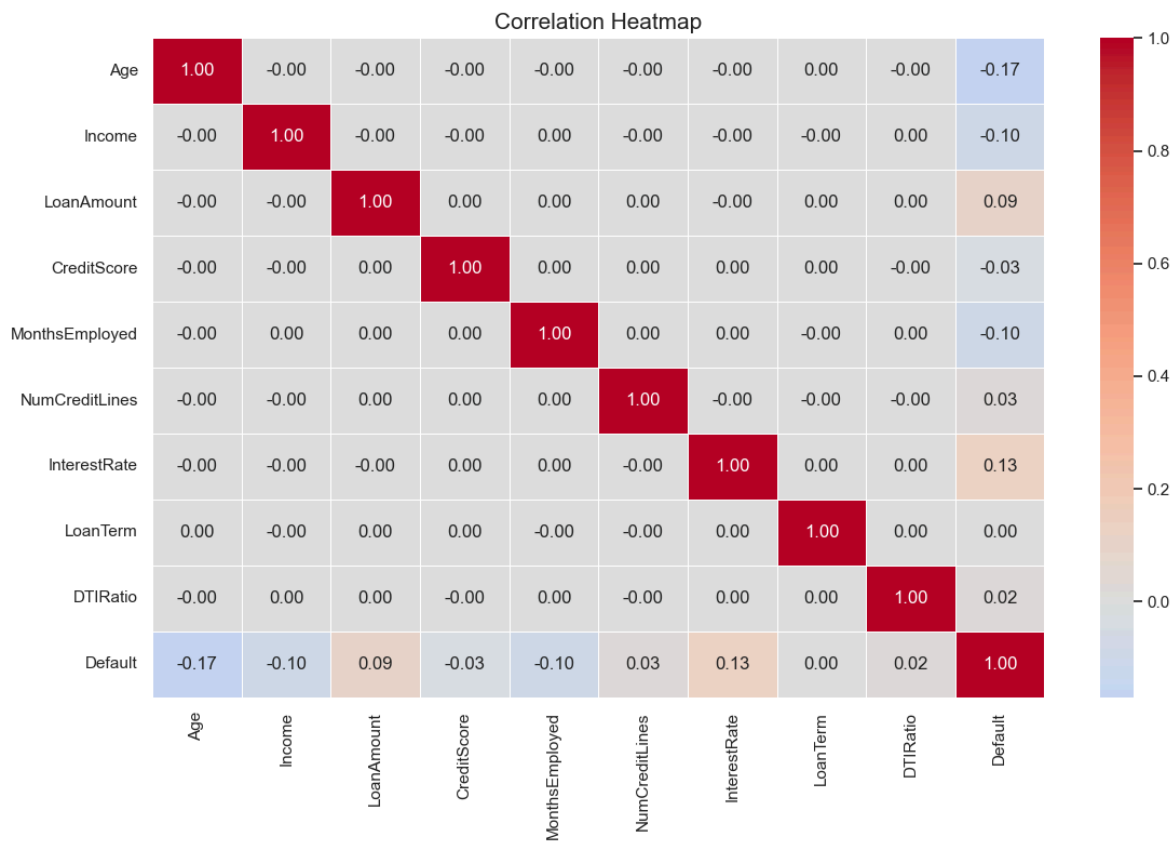
A boxplot shows the spread of data and highlights outliers (extreme values).

In [19]:
```
fig, axes = plt.subplots(3, 3, figsize=(16, 10))
axes = axes.flatten()

for i, col in enumerate(num_cols):
    axes[i].boxplot(df[col].dropna())
    axes[i].set_title(col)
    axes[i].set_ylabel('Value')

plt.suptitle('Boxplots — Checking for Outliers', fontsize=15)
plt.tight_layout()
plt.show()
```



Correlation Heatmap

Shows how strongly each numerical feature is related to the others.

- **Close to 1** => strong positive relationship
- **Close to -1** => strong negative relationship
- **Close to 0** => no relationship

In [23]:
```
# Select only numerical columns + target
corr_data = df[num_cols + ['Default']].corr().round(2)

plt.figure(figsize=(12, 8))
sns.heatmap(corr_data,
            annot=True,        # show numbers inside the boxes
            fmt='.2f',         # round to 2 decimal places
            cmap='coolwarm',   # red = positive, blue = negative
            center=0,
            linewidths=0.5)
plt.title('Correlation Heatmap', fontsize=15)
plt.tight_layout()
plt.show()
```